EL2805 Reinforcement Learning

Exercise Session 2

November 3rd 2025

Division of Decision and Systems Control
School of Electrical Engineering
KTH Royal Institute of Technology

Instructions.

- These exercises are for your practice, and you do not need to submit solutions. We encourage you to work on them before the corresponding exercise session on Nov 5^{th}. The materials for the 3^{rd} exercise session will be available on Nov 17^{th}.
- In each exercise session, we will go through a selection of topics chosen by the teaching assistant and participating students.
- Homework and labs will be based on concepts covered in lectures and exercise sessions. These materials are excellent preparation for your graded assignments!
- There is no need to memorize equations - relevant materials will be provided during the exam. Focus on concepts and understanding!
- The materials cover a wide range of topics, so feel free to focus on what interests you most. However, to develop a well-rounded understanding of RL, it is important that you engage with both theoretical and practical exercises.
- We use green boxes to highlight established results and definitions, yellow for problems that require your solutions, and pink for high-level remarks. Exercises marked with a ★ are more challenging and intended for students seeking an extra challenge.
- We value your feedback on the materials provided. If you have any questions, concerns, or suggestions, do not hesitate to reach out to any of the TAs.

Averaging vs Bootstrapping: The Middle Way

1.1 [read] Refreshing the basics of MC and TD learning

In this exercise we focus on Monte Carlo (MC) methods and Temporal-Difference (TD) learning. We begin by revisiting the core concepts behind these two methods. Here, we consider the **prediction/evaluation** problem, where our goal is to estimate the value function for a **fixed policy** π. Since the policy is fixed, we will omit π in our notation throughout this section.

Recall that the value function is defined as $V(s) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t r_t | s_0 = s\right]$, where r_t are the rewards obtained while following fixed policy π. Both MC and TD methods aim to estimate the value function V through interaction with the environment, collecting samples in the form $(s_t, r_t, s_{t+1}, r_{t+1}, \ldots)$.

MC methods estimate state values by averaging returns observed across episodes:

For each episode i, set $G_{T+1,i} = 0$ and $G_{t,i} = r_{t,i} + \gamma G_{t+1,i}$ for $t = T, T-1, \ldots, 1$.
Set number of visits to state s: $C_i(s) = \sum_{j=1}^{i} \sum_{\ell=1}^{T} 1_{(s_{\ell,j}=s)}$ and $\forall s_{t,i} \notin \{s_{1,i}, \ldots, s_{t-1,i}\}$:

$$V^{(i)}(s_{t,i}) = V^{(i-1)}(s_{t,i}) + \frac{1}{C_i(s_{t,i})}(G_{t,i} - V^{(i-1)}(s_{t,i})) \qquad \text{(MC update)}$$

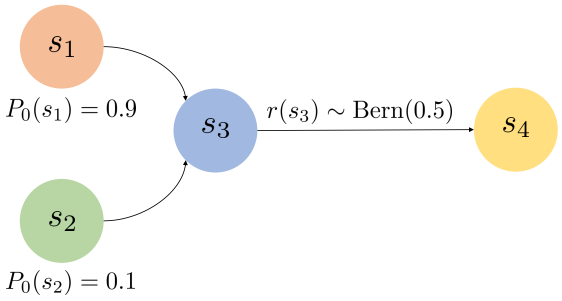
In contrast, TD methods use bootstrapping, making updates for $t = 0, 1, \ldots$ as follows:

$$V^{(t+1)}(s) = V^{(t)}(s) + \mathbf{1}_{(s_t=s)}\alpha_t(\underbrace{r_t + \gamma V^{(t)}(s_{t+1}) - V^{(t)}(s_t)}_{=\delta_t(\text{TD}-\text{error})}) \qquad \text{(TD update)}$$

Both updates refine the estimate $V^{(t+1)}$ as a weighted combination of the previous estimate $V^{(t)}$ and a target: G_t (for MC), and $r_t + \gamma V^{(t)}(s_{t+1})$ (for TD).

1.2 [solve] Getting familiar with MC and TD(0) methods

In this section, we will apply MC and TD methods on simple MDPs, inspired by examples in [1]. First, consider the MDP shown in the left figure, consisting of four states and deterministic transitions. All states yield rewards of 0, except for transitions from s_3 where $r(s_3) \sim \text{Bern}(0.5)$, a Bernoulli random variable with parameter 0.5 (which equals 0 half of the time and 1 the other half). The initial state is s_1 with probability 0.9, and s_2 with probability 0.1, while s_4 is an absorbing state.



Exercise 1.1 You observe the following four episodes (state, reward, next state, next reward, ...): $(s_1, 0, s_3, 1, s_4, 0)$, $(s_1, 0, s_3, 0, s_4, 0)$, $(s_1, 0, s_3, 1, s_4, 0)$, $(s_2, 0, s_3, 0, s_4, 0)$.
Assume zero initialization and $\alpha_i = 1/i$ for TD, where i is the episode index ($i = 1, \ldots, 4$).
Determine:

(a) the values estimated by Monte Carlo,
(b) the values estimated by TD,
(c) the true state values V,
(d) an intuitive explanation for the difference in the estimate of $V(s_2)$ between the two methods,
(e) what happens if you keep iterating both algorithms over these four episodes.

One significant difference between MC and TD methods is their bias-variance trade-off. MC methods provide an unbiased empirical estimate of the value function, though with higher vari-

ance. TD methods, on the other hand, introduce bias by targeting $r_t + \gamma V^{(t)}(s_{t+1})$, rather than $r_t + \gamma V(s_{t+1})$. In the next exercise, we explore this effect.

Exercise 1.2 Consider the MDP shown in Figure 1. Show that the variance of the TD targets for $V(s_2)$ decreases as more samples are collected, whereas the variance of the MC targets remains constant.

⚠Hints

Recall that $\mathrm{Var}(X) = p(1-p)$ for $X \sim \mathrm{Bern}(p)$, and $\mathrm{Var}(X) = np(1-p)$ for $X \sim \mathrm{Bin}(n, p)$.

Next, consider the MDP shown in Figure 2. Here, after state s_3, a long sequence of states follows without any rewards until finally, at state s_{99}, a reward of 1 is received.

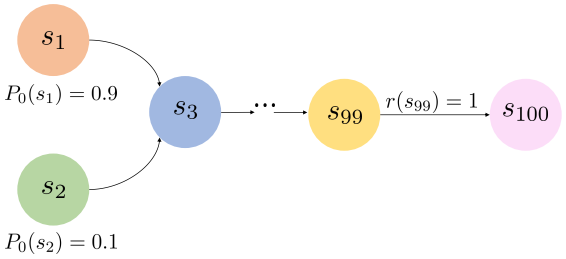


Figure 2: MDP with long trajectories

Exercise 1.3 For the MDP shown in Figure 2, estimate how many steps each method requires to accurately approximate $V(s_2)$, assuming a learning rate of $\alpha = 1$.

As we have observed, both methods have advantages and disadvantages, and the choice of method may depend on the specific MDP.

This raises the question: can we devise a method that integrates the benefits of both? We explore this in the following section.

1.3 [solve] TD-Learning with Eligibility Traces

Eligibility traces combine ideas from TD and MC methods. They form a bridge between updating after each step (TD) versus only after the entire episode (MC). We start by defining:

$$G_{t:t+n} = r_t + \gamma r_{t+1} + \cdots + \gamma^{n-1} r_{t+n-1} + \gamma^n V^{(t)}(s_{t+n}) \qquad (n\text{-return})$$

Observe that for $n = 1$, this reduces to the one-step TD update, while $n = T - t$ corresponds to the full MC return. By choosing n somewhere in between, we can balance bias and variance in learning. To combine all these possibilities, we define the λ-return, which averages over all possible n-step returns with exponentially decreasing weights:

$$G_t^\lambda = (1-\lambda) \sum_{n=1}^{T-t-1} \lambda^{n-1} G_{t:t+n} + \lambda^{T-t-1} G_{t:T} \qquad (\lambda\text{-return})$$

After completing an episode and collecting all rewards, the value function is updated $\forall t = 0, \ldots, T$:

$$V^{(t+1)}(s) = V^{(t)}(s) + \alpha(G_t^\lambda - V^{(t)}(s))\mathbf{1}_{(s=s_t)} \qquad (\text{offline } \lambda \text{ update})$$

TD(λ) algorithm. TD(λ) learns the *same* value function online, during the episode, by approximating the λ-returns. The update equation looks almost identical to one-step TD, but includes an additional multiplicative term - the eligibility trace:

$$V^{(t+1)}(s) = V^{(t)}(s) + \alpha_t z_t(s)(r_t + \gamma V^{(t)}(s_{t+1}) - V^{(t)}(s_t)) \qquad (\text{TD}(\lambda) \text{ update})$$

The eligibility trace $z_t(s)$ acts like a form of short-term memory: when a state is visited, its trace increases and then gradually fades. If a TD error occurs soon after, the trace ensures that earlier states still get credit for the outcome. There are several common ways to define z_t, the most common are accumulating (or averaging) traces: $z_t(s) = \mathbf{1}_{(s_t=s)} + \gamma\lambda z_{t-1}(s)$, and replacing traces:

$z_t(s) = \max(\mathbf{1}_{(s_t=s)}, \gamma\lambda z_{t-1}(s))$. The parameter λ controls how quickly traces decay - smaller λ results in faster forgetting, larger λ in longer memory. For more details, check Section 12 in [2].

In next exercise you will show the equivalence of these views of λ-learning under certain conditions:

★ **Exercise 1.4** Show that the total updates made by the offline λ-return algorithm are equal to those made by the online TD(λ) algorithm (using averaging eligibility traces) - provided we accumulate the updates over the entire episode but apply them only once at the end ($T = \infty$). In other words, show that:

$$\sum_{t=0}^{\infty}(G_t^\lambda - V(s))\mathbf{1}_{(s=s_t)} = \sum_{t=0}^{\infty} z_t(s)(r_t + \gamma V(s_{t+1}) - V(s_t))$$

⚠Hints

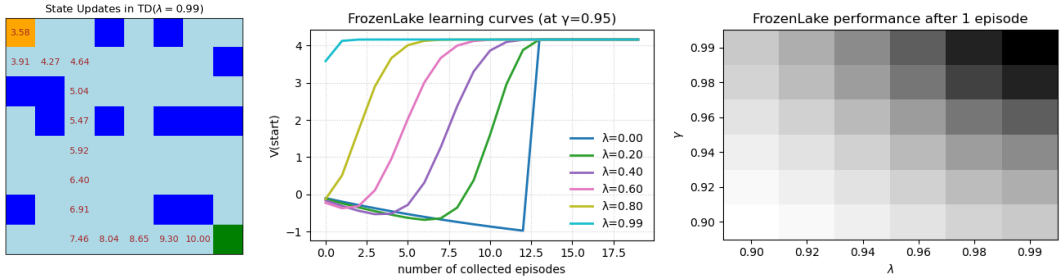
1. Starting from the definition of averaging traces, show: $z_t(s) = \sum_{k=0}^{t}(\gamma\lambda)^{t-k}\mathbf{1}_{(s=s_k)}$.
2. Show that each n-step return can be expressed using TD errors as: $G_{t:t+n} = V(s_t) + \sum_{\ell=0}^{n-1}\gamma^\ell\delta_{t+\ell}$.
3. Using result from 2. show that $G_t^\lambda = V(s_t) + \sum_{k=0}^{\infty}(\lambda\gamma)^k\delta_{t+k}$.

1.4 [code] Tracking the propagation of TD(λ) errors

In the provided script `TD_lambda.py`, you will implement and test the TD(λ) algorithm to observe how eligibility traces affect value updates and learning dynamics in the FrozenLake environment.

You can reuse parts of the policy iteration algorithm from `PI_FrozenLake_solved.py` from the first exercise session. Your tasks are as follows:

1. Run TD(λ) with the optimal policy for $\lambda = 0.0$, 0.5, and 0.99, and note which states are updated (recreate figure left). Optionally, increase episode length (e.g., 100 steps) to see if differences between λ persist.
2. With $\gamma = 0.95$, run 20 episodes and record $V(s_0)$ for each λ. Plot the learning curves (figure center) and comment on how λ affects learning speed and stability.
3. Evaluate TD(λ) after a fixed number of episodes (1 or 20) for several γ and λ values. Summarize in a table or heatmap (figure right) and identify the best-performing parameters.
4. Repeat the tasks using a non-optimal policy that chooses random actions with probability 0.1. Compare results to the optimal policy and discuss how this affects parameter selection.



★ **Further reading.** Concepts similar to λ-learning were used in one of the seminal RL papers: [3].

References

[1] Csaba Szepesvári. *Algorithms for reinforcement learning.* 2009.

[2] R. Sutton and A. Barto. *Reinforcement learning: An introduction.* MIT press, 2020.

[3] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel. High-dimensional continuous control using generalized advantage estimation. 2015.